

1. Data Wrangling

Refer “weight-height” dataset and perform following data wrangling operations

1. Scan all variables for missing values and inconsistencies. If there are missing values and/or inconsistencies, use any of the suitable techniques to deal with them.
2. Scan all numeric variables for outliers. If there are outliers, use any of the suitable techniques to deal with them.
3. Apply data transformations on at least one of the variables. The purpose of this transformation should be one of the following reasons: to change the scale for better understanding of the variable, to convert a non-linear relation into a linear one, or to decrease the skewness and convert the distribution into a normal distribution.

```
# Import necessary libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from scipy import stats
```

```
# Step 1: Load the dataset
```

```
df = pd.read_csv("weight-height.csv")
```

```
# Display basic information about the dataset
```

```
print("Dataset Info:")
```

```
print(df.info())
```

```
print("\nFirst 5 rows of the dataset:")
```

```
print(df.head())
```

```
# Step 2: Scan for missing values
```

```
print("\nCheck for Missing Values:")
```

```
print(df.isnull().sum())
```

```

# Handle missing values if present (e.g., fill with mean or median)
df.fillna(df.mean(), inplace=True)

# Step 3: Scan numeric variables for outliers

# Define a function to check for outliers using Z-scores
def detect_outliers(df, col_name):
    z_scores = stats.zscore(df[col_name])
    outliers = np.where(np.abs(z_scores) > 3) # Outliers have Z-scores > 3
    return outliers

# Check for outliers in 'Weight' and 'Height' columns
outliers_weight = detect_outliers(df, 'Weight')
outliers_height = detect_outliers(df, 'Height')

print("\nOutliers in 'Weight' at indices:", outliers_weight[0])
print("Outliers in 'Height' at indices:", outliers_height[0])

# Handling outliers: Capping the outliers (we use 95th percentile to cap the outliers)
df['Weight'] = np.where(df['Weight'] > np.percentile(df['Weight'], 95),
                        np.percentile(df['Weight'], 95),
                        df['Weight'])
df['Height'] = np.where(df['Height'] > np.percentile(df['Height'], 95),
                        np.percentile(df['Height'], 95),
                        df['Height'])

# Step 4: Apply Data Transformation (e.g., Log Transformation)

# Check skewness before transformation
print("\nSkewness of 'Weight' before transformation:", df['Weight'].skew())

```

```
print("Skewness of 'Height' before transformation:", df['Height'].skew())

# Apply a log transformation to 'Weight' (as it may be positively skewed)
df['Weight_transformed'] = np.log(df['Weight'] + 1) # Adding 1 to avoid log(0)

# Check skewness after transformation
print("\nSkewness of 'Weight' after transformation:", df['Weight_transformed'].skew())

# Step 5: Visualizations for understanding
plt.figure(figsize=(14, 7))

# Original distribution of 'Weight'
plt.subplot(1, 2, 1)
sns.histplot(df['Weight'], kde=True)
plt.title("Original Weight Distribution")

# Transformed distribution of 'Weight'
plt.subplot(1, 2, 2)
sns.histplot(df['Weight_transformed'], kde=True)
plt.title("Transformed Weight Distribution")

plt.show()

# Step 6: Final Dataset Preview
print("\nFinal Dataset after Wrangling:")
print(df.head())
```